Mid-Year Review

A problem-solving method, a topic map, and a mock contest

Learning goals

- Connect the 17 topics so far into **one toolbox**
- Use a repeatable **method** for an unseen problem
- Turn constraints into a **target complexity** in seconds
- Recognise **signals** that point to a technique
- Sit a 3-hour **mock contest** and learn from the debrief
- Build an **upsolving** habit for the second half of the year

Where we are

Weeks	Block	Core tools
1–3	Foundations	judges, STL, complexity, brute force
4–7	Array techniques	sorting, greedy, binary search, two pointers, prefix sums
8–11	Search & math	backtracking, bitmasks, number theory, modular arithmetic
12–13	Data structures	stacks/queues, monotonic structures, heaps, sets/maps
14–15	Dynamic programming	1D DP, knapsack, LIS, LCS, grid DP
16–17	Graphs	BFS/DFS, components, bipartite, shortest paths

The method (1/2): understand

1. **Read everything** — statement, input format, constraints, notes
2. **Restate** the task in one sentence, in math, without the story
3. **Work the samples by hand** — do you get the same output?
4. **Make your own small cases**, especially edge cases: n = 1, all equal, all negative, disconnected graph
5. **Read the constraints** → target complexity (next slide)

> Most wrong answers come from solving a slightly different problem.

Constraints → target complexity

~10^8 simple operations per second.

n up to	Target	Typical techniques
10–12	O(n!)	permutations, backtracking
20–25	$O(2^n \cdot n)$	bitmask enumeration, meet-in-the-middle
100–500	$O(n^3)$	Floyd-Warshall, interval DP
5000	$O(n^2)$	2D DP, all pairs
$2 \cdot 10^5 – 10^6$	O(n log n) / O(n)	sorting, binary search, heaps, BFS, prefix sums
$10^9 – 10^{18}$	O(log n) / O(√n)	math, binary search on answer, fast power

The method (2/2): solve

6. **Brute force first** (in your head or on paper) — what is slow about it?
7. **Look for structure**: monotonicity? sortedness? overlapping subproblems? graph?
8. **Match signals** to techniques (next slides)
9. **Prove or at least test** the idea on your small cases *before* coding
10. **Code cleanly**, then test samples + your edge cases, then submit

If stuck > 15 min: reread the statement, try tiny cases exhaustively, change the viewpoint (sort? reverse? complement?).

Signals → techniques (1/2)

You see...	Think...
"minimum maximum", "smallest T such that..."	binary search on the answer (W5)
sorted array, pairs with sum / difference	two pointers (W6)
many range-sum queries, no updates	prefix sums (W7)
many range additions, then read	difference array (W7)
"next greater", "largest rectangle"	monotonic stack (W12)
sliding window max/min	monotonic deque (W12)
repeatedly take the best item	priority queue (W13)

Signals → techniques (2/2)

You see...	Think...
"count ways", "min cost", choices with overlap	DP (W14–15)
subset under a budget	knapsack (W15)
two strings / sequences	prefix × prefix DP (W15)
n ≤ 20, "choose a subset"	bitmasks (W9)
"mod 10^9+7", "nCr"	modular arithmetic (W11)
grid, maze, "minimum moves"	BFS (W16)
weighted, "cheapest route"	Dijkstra (W17)
"can be split into two groups"	bipartite (W16)

Debugging checklist (before you resubmit)

- **WA**: overflow? `int` vs `long long`; off-by-one; uninitialised / not-reset arrays between tests; reading input wrong; special cases n = 1
- **TLE**: complexity really OK? `endl` / no fast I/O; copying vectors by value; `map` where array works; infinite loop
- **RE**: out-of-bounds index; recursion depth; division by zero; empty container `.top()` / `.front()`
- **MLE**: 2D array too large; storing all states when rolling suffices

Week 35 adds stress testing: brute force + random generator + comparison.

Mock contest: rules

- **3 hours**, individual, 8 problems, roughly increasing difficulty
- Any language reference allowed; no searching for solutions
- Penalty: ICPC-style (solved count, then time + 20 min per wrong try)
- Keep a log: when you read each problem, when you submitted, why it failed
- After the contest: **debrief** (today) and **upsolve** (this week)

Mock contest: problem set

#	Problem	Topic hint (revealed after)
A	CF 1352C K-th Not Divisible by n	math
B	CSES 1640 Sum of Two Values	sorting / two pointers
C	CSES 1620 Factory Machines	binary search on answer
D	CSES 1662 Subarray Divisibility	prefix sums + counting
E	CSES 1746 Array Description	DP
F	AtCoder ABC 151 D Maze Master	BFS from every cell
G	CF 1272E Nearest Opposite Parity	multi-source BFS, reversed graph
H	CF 1201C Maximum Median	sorting + binary search

Debrief A: K-th Not Divisible by n

Among 1..x, exactly `x - [x/n]` numbers are not divisible by n.

- Every block of n consecutive numbers contributes n − 1 good numbers
- k = q·(n − 1) + r … careful when r = 0
- Clean closed form: answer = `k + (k - 1) / (n - 1)`

Check n = 3, k = 7: 1 2 4 5 7 8 **10** → 7 + 6/2 = 10 ✓

Lesson: count "how many ≤ x" and invert. (Binary search on x also works!)

Debrief B & C

B — Sum of Two Values: store (value, index), sort, two pointers from both ends; or one pass with `map<value, index>`. O(n log n).

C — Factory Machines: "minimum time T such that $\Sigma \lfloor T / k_i \rfloor \geq t$".

- Predicate is **monotone** in T → binary search
- Upper bound: `min(k) · t` ($\leq 10^{18}$)
- Overflow: Σ can exceed 10^{18} → **stop summing once ≥ t**

Code C: Factory Machines

```
auto enough = [&](ll T) {
    ll made = 0;
    for (ll x : k) {
        made += T / x;
        if (made >= t) return true;     // early exit prevents overflow
    }
    return false;
};
ll lo = 0, hi = *min_element(k.begin(), k.end()) * t;
while (hi - lo > 1) {                   // !enough(lo), enough(hi)
    ll mid = lo + (hi - lo) / 2;
    if (enough(mid)) hi = mid; else lo = mid;
}
cout << hi << "\n";
```

Debrief D: Subarray Divisibility

Sum of a[l..r] = P[r] − P[l−1]. Divisible by n ⇔ P[r] ≡ P[l−1] (mod n).

→ count pairs of prefixes with equal remainder.

```
cnt[0] = 1;                                  // empty prefix
for (...) {
    pre = ((pre + a) % n + n) % n;           // negative a!
    ans += cnt[pre]++;
}
```

Pitfall: `%` of a negative number is negative in C++. O(n).

Debrief E: Array Description

`dp[i][v]` = ways to fill x[0..i] with x[i] = v.

```
dp[i][v] = dp[i-1][v-1] + dp[i-1][v] + dp[i-1][v+1]   if x[i] ∈ {0, v}
dp[i][v] = 0                                          otherwise
```

- $n \le 10^5$, $m \le 100$ → 10^7 states, O(1) each ✓
- Pad indices 0 and m+1 with zeros to avoid bounds checks
- Roll the i dimension: two arrays of size m + 2

Debrief F & H

F — Maze Master (H, W ≤ 20): maximum over all start cells of (max BFS distance). 400 BFS × 400 cells = $1.6 \cdot 10^5$. Constraints say "just brute force it with BFS".

H — Maximum Median: sort; only the upper half matters. Binary search on the answer M: cost(M) = Σ over i ≥ n/2 of max(0, M − a_i). Feasible if cost ≤ k. Watch `long long`.

Lesson: small constraints are a **hint**, not a trap.

Debrief G: Nearest Opposite Parity

From i you may jump to i ± a[i]. For each i: fewest jumps to reach any j with a[j] of opposite parity.

- n BFS's (one per start) = $O(n^2)$ ×
- **Reverse the edges** and run **multi-source BFS** from all targets at once
- Two runs: sources = all odd values (answers for even starts), then all even values

O(n). The trick "reverse the graph, start from the targets" appears constantly.

Code G: reversed multi-source BFS

```
for (int i = 0; i < n; i++) {
    if (i - a[i] >= 0) radj[i - a[i]].push_back(i);
    if (i + a[i] < n)  radj[i + a[i]].push_back(i);
}
for (int par = 0; par < 2; par++) {
    vector<int> dist(n, -1); queue<int> q;
    for (int j = 0; j < n; j++)
        if (a[j] % 2 == par) { dist[j] = 0; q.push(j); }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : radj[u])
            if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }
    }
    for (int i = 0; i < n; i++) if (a[i] % 2 != par) ans[i] = dist[i];
}
```

After the contest: upsolving

- Solve **every** problem you didn't solve, within a week
- For each failed attempt, write one line: *what was the bug / missing idea?*
- Keep a personal **mistake log** — the same 5 bugs cause 80% of your WAs
- Re-solve a problem from scratch 3 days later without looking
- Rate yourself per topic: green / yellow / red → plan the next 2 weeks

Practice: review set

- Weeks 4–7: CSES 1090 Ferris Wheel, CSES 1085 Array Division, CSES 1641 Sum of Three Values, CSES 1661 Subarray Sums II
- Weeks 9–13: CSES 1623 Apple Division, CSES 1095 Exponentiation, CSES 1645 Nearest Smaller Values, CSES 1164 Room Allocation
- Weeks 14–17: CSES 1634 Minimizing Coins, CSES 1145 Increasing Subsequence, CSES 1194 Monsters, CSES 1195 Flight Discount
- Virtual contest: any recent Codeforces Div. 3 round

Summary

- One method: understand → constraints → brute force → structure → signals → verify → code
- Constraints tell you the complexity; the complexity narrows the technique
- Most bugs are boring: overflow, bounds, resets, misread statements
- Mock contest = measurement; **upsolving** = learning
- Next week: Disjoint Set Union & Minimum Spanning Trees